Proposal

Bayesian F1 Grid Gap Analysis

Shahrukh Islam Prithibi (47610068), Khue Do

2026-03-06

Packages

```
# install.packages("f1dataR")
# install.packages("dplyr")
```

Team

Shahrukh Islam Prithibi, Khue Do

Github Repo Link:

https://github.com/khuedo22/stat-405-final-project.git

Datasets

Primary Dataset

We use the `f1dataR` R package which pulls data directly from the Jolpica/Ergast F1 API. The dataset contains race results for every F1 season from 1950–2024. For our analysis we focus on the 2019–2024 seasons. Each row corresponds to one driver's result in one race.

URL: https://github.com/SCasanova/f1dataR

The key variables we use are:

- `driver_id` — unique identifier for the driver
- `constructor_id` — the team the driver races for

- `grid` — starting grid position (1–20)
- `gap` — finishing time behind the race winner in seconds
- `position` — finishing position
- `status` — whether the driver finished or retired (used for filtering)
- `circuit` — the circuit where the race took place (from schedule)
- `race_name` — the name of the Grand Prix (from schedule)

```
library(f1dataR)
library(dplyr)

# Load schedule to get circuit and race name
schedule <- load_schedule(season = 2023)

sample <- load_results(season = 2023, round = 1) |>
  mutate(
```

```r
    season        = 2023,
    round         = 1,
    race_name     = schedule$race_name[1],
    circuit       = schedule$circuit_id[1],
    top_speed_kph = as.numeric(top_speed_kph)
  )

head(sample)
```

```
## # A tibble: 6 x 16
##   driver_id constructor_id points position grid  laps  status gap   fastest_rank
##   <chr>     <chr>          <chr>  <chr>    <chr> <chr> <chr>  <chr> <chr>
## 1 max_vers~ red_bull       25     1        1     57    Finis~ 1:33~ 6
## 2 perez     red_bull       18     2        2     57    Finis~ +11.~ 7
## 3 alonso    aston_martin   15     3        5     57    Finis~ +38.~ 5
## 4 sainz     ferrari        12     4        4     57    Finis~ +48.~ 14
## 5 hamilton  mercedes       10     5        7     57    Finis~ +50.~ 10
## 6 stroll    aston_martin   8      6        8     57    Finis~ +54.~ 9
## # i 7 more variables: fastest <chr>, top_speed_kph <dbl>, time_sec <dbl>,
## #   season <dbl>, round <dbl>, race_name <chr>, circuit <chr>
```

Backup Dataset

Our backup is a Kaggle dataset containing F1 race results as pre-downloaded CSV files, available via our GitHub repository.

URL: https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020

The relevant files are `results.csv`, `races.csv`, `drivers.csv`, and `constructors.csv` which join on `raceId`, `driverId`, and `constructorId` respectively. Unlike the primary dataset, this source does not contain a `gap` column directly. Instead, we engineer it from the `milliseconds` column in `results.csv`, which records each driver's total race time in milliseconds. We compute the gap by subtracting the winner's race time from each driver's race time and converting to seconds:

```
gap = (driver_milliseconds - winner_milliseconds) / 1000
```

```r
base <- "https://raw.githubusercontent.com/khuedo22/stat-405-final-project/main/data_backup/"

results      <- read.csv(paste0(base, "results.csv"))
races        <- read.csv(paste0(base, "races.csv"))
drivers      <- read.csv(paste0(base, "drivers.csv"))
constructors <- read.csv(paste0(base, "constructors.csv"))

f1_backup <- results |>
  left_join(races |> rename(race_name = name),       by = "raceId") |>
  left_join(drivers,                                  by = "driverId") |>
  left_join(constructors |> rename(team_name = name), by = "constructorId") |>
  select(season = year, round, raceId, driverId,
         driver = surname, team = team_name,
         race_name, grid, position, points,
         milliseconds) |>
  filter(season == 2023) |>
  mutate(milliseconds = as.numeric(milliseconds)) |>  # force to numeric
  group_by(raceId) |>
  mutate(
    winner_ms = min(milliseconds, na.rm = TRUE),
```

```r
    gap = (milliseconds - winner_ms) / 1000
  ) |>
  ungroup()

head(f1_backup)
```

Question of interest

Does grid position affect a driver's finishing gap to the race winner, and does this effect vary by circuit?

Grid position in F1 is set during qualifying: the faster you go, the further up the grid you start on race day. Starting further back generally means more cars to get past and a higher chance of getting stuck in traffic, which should translate to finishing further behind the winner. But how much this actually matters is probably not the same everywhere. At a place like Monaco, the streets are so narrow that overtaking is nearly impossible, so where you start is pretty much where you finish. Monza is the opposite: long straights and DRS zones mean cars can pass each other more freely, so a driver starting 10th still has a realistic shot at a top finish.

This leads to our question:

> *Does grid position affect a driver's finishing gap to the race winner, and does this effect vary by circuit?*

We want to estimate how much of a time penalty a driver faces for each grid position they start behind the front and whether that penalty is bigger at some circuits than others. We think Bayesian methods are a natural fit here since each circuit only has one race per season, so we are working with limited data per group. A hierarchical model lets circuits with less data borrow information from the broader pool of circuits rather than being estimated in isolation, which gives more reliable estimates especially for newer circuits on the calendar.

Methodology

The parameter of interest will be the coefficient for a driver's starting position. We plan to fit a Bayesian generalized regression model. Since our response variable (`position`) is a discrete value from 1-10, we might model this either as a Categorical distribution. The function for the model is $\alpha \times$ grid. p_i will the the logit of the function. The parameters of interest are the coefficient for grid position. This could follow a normal distribution. We can build a hierarchial model for the parameters of the normal distribution for the coefficient.

Plan

- data preprocessing
- EDA
- define some plausible models (distributions of data and parameters)
- run model in STAN
- build credible intervals and check whether they are well-calibrated (by predicting the next data point)
- model rank histogram and trace plots, other diagnostics
- iterate if needed